From Patch to Affected Releases: Predicate-Guided Vulnerability-State Verification across Software Versions

Anonymous Authors

***Abstract*—Identifying which released versions are affected by a disclosed vulnerability is essential for vulnerability management, dependency maintenance, and downstream risk assessment. Existing approaches often infer affected version ranges from advisory text, fixing commits, or vulnerability-inducing commits, but such interval-based reasoning is fragile under backports, release-branch divergence, refactorings, partial fixes, and vulnerability-preserving code movement. This paper formulates CVE affected-version identification as release-level vulnerability-state verification. Given a CVE fixing commit and a repository's release history, our key insight is that each release tag should be classified by verifying whether the root-cause vulnerability predicate still holds in that release, rather than by its position in a commit interval. We present VulnVersion, a predicate-guided approach that extracts root-cause anchors and vulnerability/fix predicates from patches, collects repository-local evidence across release tags, and produces tag-level vulnerability states through evidence aggregation and constrained LLM-assisted semantic judgment. We evaluate VulnVersion on a public benchmark for vulnerability-affected version identification and compare it against advisory-based, commit-range, SZZ-style, patch-similarity, and LLM-only baselines.**

***Index Terms*—software vulnerability, affected-version identification, software evolution, program analysis, large language models**

I. Introduction

Modern software ecosystems depend on vulnerability databases to decide whether a deployed package version is exposed to a known CVE. In practice, however, the public metadata attached to a CVE is often incomplete, imprecise, or difficult to align with the actual release history of an open-source project. A maintainer, scanner, or downstream user may know the CVE identifier and the fixing patch commits, yet still face the harder question: which released versions actually contain the vulnerable logic?

This paper targets the affected-version identification problem studied by Chen et al. [1]. Given a reported vulnerability, a project repository, the project release versions, and patch evidence such as fixing commits, the goal is to identify the subset of released versions where the vulnerable logic remains present. This setting is more demanding than simply finding a vulnerability-inducing commit. A vulnerability-inducing commit is a historical event; an affected-version set is a state predicate over released tags. The mapping between the two can be broken by multi-branch development, backports, partial fixes, refactoring, cherry-picks, noisy patches, and add-only fixes.

Existing techniques broadly follow two families. Tracing-based techniques inherit the SZZ-style pipeline: choose changed statements from a fixing patch, trace them back to candidate inducing commits, and infer affected versions from commit reachability. Matching-based techniques instead extract patch-derived signatures or vulnerable-code representations and test whether similar logic appears in historical versions. The recent benchmark study "How Far Are We?" reports that both families remain insufficient under large-scale, version-level evaluation, with the best individual tool below 45.0% CVE-level exact accuracy [*NEEDS TABLE REPAIR*] [1]. Its error analysis points to several recurring causes: noisy patches, semantic mismatch between the fixed lines and the root cause, shallow presence verification, add-only patches, multi-file changes, and multi-branch evolution.

The broader reference corpus reinforces this gap from several directions. SZZ-family work improves commit localization with semantic slicing, context-aware LLM assessment, temporal knowledge graphs, and agentic repository search [2]–[6], but the native output remains a commit-level explanation. Recurring-vulnerability detectors scale vulnerable-code search through staged filtering, clone signatures, hashes, and function fingerprints [7]–[12], but code resemblance is not by itself a version-level vulnerability verdict. Direct vulnerable-version methods and developer-log approaches move closer to our target [13]–[15], yet they still expose the need for explicit patch-family filtering, root-cause evidence extraction, branch-aware version reasoning, and resource-aware semantic verification.

These observations motivate a different formulation. Instead of treating the affected-version set as a by-product of a commit interval, VulnVersiontreats each released version as a vulnerability state to be verified in repository context. The key idea is not to run an agent over every tag blindly. Rather, VulnVersionseparates deterministic repository planning from semantic vulnerability judgment. Deterministic stages construct patch-family evidence, root-cause evidence, release-line structure, and candidate probe tags. The agent is used only for selected tag verdicts, where it receives compressed evidence and must decide whether the vulnerable condition is present in that repository state. This boundary is important for cost, reproducibility, and auditability.

The current paper draft deliberately leaves the final algorithmic details of VulnVersionas structured placeholders. The system design documents define three high-level modules: Step1 performs fix-family and patch-chunk semantic

filtering; Step2 extracts root-cause-level vulnerability existence evidence; Step3 builds a repository-aware version-line plan and verifies selected tags. These modules are grounded in the local design artifacts, but the final method, ablations, and full results are still under development. We therefore separate confirmed problem framing and evaluation design from claims that require future experiments.

This draft makes the following intended contributions: leftmargin=*

- We formulate CVE affected-version identification as repository-grounded per-tag vulnerability state verification, separating it from vulnerability-inducing commit localization and patch-signature resemblance.
- We propose the high-level architecture of VulnVersion, which combines patch-family filtering, root-cause evidence extraction, repository-aware release-line planning, and scoped agent-assisted tag verdicts.
- We define an ICSE-style evaluation plan using the 1,128-CVE, 9-project benchmark from "How Far Are We?" as the main dataset and baseline source, with CVE-level, version-level, robustness, ablation, and resource-cost metrics.
- We synthesize design lessons from affected-version identification, SZZ/BIC localization, recurring-vulnerability detection, and LLM/agent-based software-engineering systems into a concrete upgrade plan for the VulnVersionpaper.

The rest of this paper is organized as follows. Section II introduces the task and the limitations of existing approaches. Section IV presents the current VulnVersiondesign placeholder. Section VI specifies the planned evaluation protocol. Section VIII reviews related work. Section VII discusses threats to validity.

II. Background and Problem Statement

A. Task Definition

Let P be a target open-source project, $V = \{v_1, \ldots, v_n\}$ be the set of released versions or tags, and c be a CVE with one or more fixing commits. The goal is to recover:

$$V_{aff}(c, P) = \{v \in V \mid \text{the vulnerable logic of } c \text{ is present in } v\}.$$

The input setting follows the baseline paper [1]: a reported vulnerability, a project, released versions, and patch evidence such as fixing commits. The output is a set of affected released versions, not a vulnerability-inducing commit and not merely a vulnerable function.

This distinction matters because version-level vulnerability exposure is not always a simple interval over the commit history. A project may maintain multiple release lines, backport a fix to selected branches, apply semantically equivalent but syntactically different patches, or remove and reintroduce relevant code through refactoring. Therefore, a method that finds a plausible inducing commit can still misidentify the affected releases.

B. Tracing-Based and Matching-Based Baselines

The baseline corpus divides direct affected-version identification methods into tracing-based and matching-based families [1]. Tracing-based methods include VCCFinder [16], V-SZZ [13], Lifetime [17], SEM-SZZ [3], TC-SZZ [18], and LLM4SZZ [2]. Their common structure is to trace from a fixing patch toward candidate inducing commits and then infer affected versions from history.

Matching-based methods include ReDeBug [9], VUDDY [12], MOVERY [8], V1SCAN [10], FIRE [7], and VULTURE [11]. These methods construct vulnerability-related signatures or representations and search for them in target code versions. They reduce dependence on a single inducing commit, but they can be brittle under refactoring, syntax-preserving edits, alternative fixes, or cases where the patch does not delete the vulnerable logic.

C. Adjacent Evidence Sources

Several adjacent lines of work provide useful evidence but do not fully solve the target task. Commit-localization methods, including classic VCC prediction, semantic SZZ variants, LLM4SZZ, AgentSZZ, and temporal-graph SZZ, improve the ability to find bug- or vulnerability-inducing commits [2]–[6], [16]. They are useful upstream signals, but they do not directly decide whether each released tag is affected. Recurring-vulnerability and vulnerable-clone detectors, including ReDeBug, VUDDY, MOVERY, V1SCAN, FIRE, and VULTURE, provide scalable code-presence evidence [7]–[12]. Their outputs are also evidence rather than final affected-version labels. Developer-log and vulnerable-version systems such as V-SZZ, TDSC, and Vercation are closest to our target [13]–[15], but their designs still motivate explicit treatment of branch history, patch semantics, LLM cost, and manual verification boundaries.

D. Failure Modes

The local reference analysis of "How Far Are We?" identifies several failure modes that directly shape VulnVersion: leftmargin=*

- **Noisy patches.** A CVE may have multiple fixing commits, and a fixing commit may include wrapper changes, backports, tests, documentation, changelog edits, refactoring, or contextual changes unrelated to the root cause.
- **Semantic mismatch.** The lines changed by the fix are not always the lines that best characterize the vulnerable condition. The root cause may span functions, files, data flow, or control-flow context.
- **Shallow presence verification.** A deleted line or a patch signature is not by itself a proof that a historical tag is vulnerable. The method must verify whether the vulnerable condition still holds in that tag.
- **Patch-type sensitivity.** Add-only patches, multi-file patches, and multi-branch fixes break assumptions used by both tracing and matching methods.

E. Design Requirements

These failure modes and adjacent evidence sources imply five requirements for VulnVersion: leftmargin=*

1) **Patch-family awareness.** The system must represent one CVE as a family of fixing commits rather than a single patch.
2) **Root-cause evidence extraction.** The system must distinguish vulnerability evidence from ordinary touched code.
3) **Repository-aware version planning.** The system must reason over release tags and branch/release-line structure without assuming a single global version order.
4) **Auditable tag verdicts.** The final affected/not-affected decision for selected tags must be tied to explicit repository evidence.
5) **Resource-aware agent use.** If an LLM/agent is used, the paper must report cost, tokens, time, tool calls, and failures rather than only accuracy.

These requirements lead to the staged design in Section IV.

III. Motivating Example

IV. VulnVersionApproach

This section describes the current high-level design of VulnVersion. It is intentionally written as a structured placeholder: it captures the design choices that are already supported by the local SystemDesign documents, while leaving implementation details, optimization choices, and final algorithmic claims as [*NEEDS METHOD DETAIL*].

A. Overview

VulnVersiontakes as input a CVE record, the target repository, the fixing commit family, release tags, and optional CVE context such as NVD description or advisory text. It outputs a predicted set of affected release tags. The framework is organized into three stages: leftmargin=*

1) **Step1: fix-family and patch semantic filtering.** Parse fixing commits, extract changed files and hunks, classify commit-level and chunk-level roles, and produce patch semantics for later reasoning.
2) **Step2: root-cause vulnerability evidence extraction.** Convert patch semantics and CVE context into root-cause-level evidence, including vulnerable sequences, fix guards, scope constraints, and evidence risks.
3) **Step3: repository-aware tag planning and verification.** Build release-line structure, select candidate probe tags, ask an agent to judge selected tags, and infer affected intervals from evidence-backed verdicts.

The corpus synthesis suggests a design rule that should be preserved as the method matures: deterministic evidence preparation should reduce the search space before expensive semantic judgment. FIRE, MOVERY, ReDeBug, V1SCAN, VULTURE, and VUDDY all use staged filtering or indexing to make vulnerability search scalable [7]–[12]; AgentSZZ and related agent papers show that scoped tool interfaces and context compression are important for keeping LLM-based repository investigation tractable [4], [6]. VulnVersionadopts this as a paper-level boundary: the agent should judge selected evidence-backed tags, not plan the entire version search.

B. Step1: Fix-Family Semantic Filtering

The first stage exists because a CVE may map to multiple fixing commits and because each fixing commit may contain irrelevant hunks. The local design document records that the baseline dataset contains 1,128 CVEs and 1,542 patches, with 68 multi-commit CVEs after local flattening. It also records 4,783 patch chunks across the local dataset processing artifacts. These figures should be rechecked before camera-ready use [*NEEDS EVIDENCE*].

Step1 is designed to output structured artifacts such as `fix_family_semantics.json`, `commit_semantics.jsonl`, `chunk_semantics.jsonl`, and `patch_semantics.json`. Its role is not to decide affected versions. Instead, it provides clean, typed, and source-referenced patch evidence to Step2. In particular, Step1 should distinguish primary fixes from supporting fixes, contextual changes, tests, documentation, wrapper commits, and refactoring noise.

C. Step2: Root-Cause-Level Evidence

Step2 extracts a root-cause-level vulnerability existence theorem, abbreviated as VET in the design documents. The intended VET representation separates five concepts:

$$\Theta = \langle S, V, F, G, C \rangle$$

where S is the vulnerability scope, V is vulnerable evidence, F is fix evidence, G is guard or applicability constraints, and C is the certificate policy. This representation is still a design placeholder [*NEEDS METHOD DETAIL*], but it encodes an important boundary: touched files and changed lines are not automatically root-cause evidence.

Step2 should output artifacts such as `root_cause_vet.json`, `vet_evidence_graph.json`, `vet_admission_input.json`, and a compatibility `rci.json`. These artifacts provide the evidence packet used by Step3 for prioritization and tag-level judgment.

This design is motivated by repeated warnings across the reference corpus: patch signatures, deleted lines, and cloned functions are useful evidence, but none of them alone proves that the vulnerability condition holds in a released version [8], [9], [12], [13], [15]. The VET artifact is therefore meant to make the required vulnerable condition explicit before any tag verdict is requested.

D. Step3: Version-Line Planning and Tag Verification

Step3 is responsible for affected-version identification. The local design document states the current main path:

release tags → release filtering → version-line graph → active-line schedu

The agent boundary is explicit: the agent judges whether a selected tag is affected; it does not choose the scan range, build the release-line graph, or plan the probing strategy.

This separation is central to VulnVersion. Deterministic planning keeps the method auditable and cost-aware; agent verdicts are reserved for cases where semantic repository evidence must be interpreted. The final Step3 design still requires experimental validation before it can be presented as a complete algorithm [*NEEDS EXPERIMENT*].

E. Expected Agent Interface

The planned agent interface is deliberately narrow. For each selected tag, the agent receives scoped evidence derived from Step1 and Step2, repository snippets from the tag under test, and a required output schema containing an `AFFECTED`, `NOT_AFFECTED`, or uncertainty-aware verdict plus evidence references. Inspired by agentic SZZ evaluations [4], [6], VulnVersionshould log token usage, tool calls, latency, and failure modes for every verdict. The exact prompt, tool API, and verdict schema remain [*NEEDS METHOD DETAIL*].

F. Memory and Self-Evolution Hooks

The broader SystemDesign materials describe typed memory, verifier-gated skill evolution, and artifact promotion as future capabilities. In this draft, these are not claimed as completed experimental components. They are positioned as optional design hooks: failed verdict traces may become failure memories, repeated repair procedures may become procedural memories, and stable verified patterns may eventually be compiled into deterministic artifacts. Any final claim about these mechanisms requires implementation and ablation evidence [*NEEDS EXPERIMENT*].

V. Implementation

VI. Evaluation Design

This section specifies the planned evaluation. It does not report final VulnVersionresults. All missing results are explicitly marked.

A. Dataset

The main dataset will be the benchmark used by "Vulnerability-Affected Versions Identification: How Far Are We?" [1]. The local reference analysis states that the benchmark contains 1,128 C/C++ vulnerabilities from nine open-source projects and 132 vulnerability types. Exact project-level table values and patch-type counts must be verified against repaired tables before final use [*NEEDS TABLE REPAIR*]. The dataset is selected because it is directly aligned with our target problem: given CVEs and patch commits, identify affected release versions.

B. Baselines

We will compare against the baselines organized in the benchmark study: leftmargin=*

- **Tracing-based:** VCCFinder [16], V-SZZ [13], Lifetime [17], SEM-SZZ [3], TC-SZZ [18], and LLM4SZZ [2].
- **Matching-based:** ReDeBug [9], VUDDY [12], MOVERY [8], V1SCAN [10], FIRE [7], and VULTURE [11].

If official runnable artifacts are unavailable or inconsistent, we will report whether baseline values are taken from the paper table or reproduced locally. No reproduced baseline claim will be made until scripts, data, and environment are verified [*NEEDS ARTIFACT*].

C. Metrics

We follow the metric structure extracted from the baseline and related papers: leftmargin=*

- **CVE-level Accuracy:** the fraction of CVEs for which the predicted affected-version set exactly equals the ground truth set.
- **CVE-level No-Miss Ratio:** the fraction of CVEs for which all ground-truth affected versions are included in the prediction.
- **Version-level Precision/Recall/F1:** flattened version-level metrics over all CVE-version pairs.
- **Resource metrics:** average time, input tokens, output tokens, total tokens, cost, agent calls, probe tags, and execution failure rate for VulnVersion[*NEEDS EXPERIMENT*].
- **Manual-workload and lower-bound metrics:** if complete FN validation is infeasible for some cases, we will report conservative lower-bound recall and manual inspection counts, following the evaluation style of developer-log based affected-version work [14].

D. Research Questions

RQ1: Overall effectiveness. How accurately does VulnVersionidentify affected versions compared with tracing-based and matching-based baselines? We will report CVE-level Accuracy, CVE-level NMR, and version-level precision/recall/F1. Result table: [*NEEDS EXPERIMENT*].

RQ2: Robustness across patch and repository conditions. How does performance vary across add-only, delete-only, mixed patches, single-file and multi-file patches, and single-branch versus multi-branch settings? Subset definitions will follow the benchmark where available. Result table: [*NEEDS EXPERIMENT*].

RQ3: Contribution of each VulnVersionstage. What is the effect of patch-family filtering, root-cause evidence extraction, release-line planning, and agent tag verdicts? Planned ablations include disabling Step1 filtering, replacing Step2 VET with raw touched-code evidence, replacing repository-aware line planning with simpler tag ordering, and replacing agent verdicts with deterministic matching. Result table: [*NEEDS EXPERIMENT*].

RQ4: Cost and failure analysis. What is the runtime, token cost, and failure rate of VulnVersion? Which cases fail because of patch noise, root-cause extraction error, release-line planning error, tag-verdict error, timeout, or missing artifacts? Result table: [*NEEDS EXPERIMENT*].

RQ5: Agent behavior and evidence use. When VulnVersioninvokes an agent, which evidence sources and tool calls drive the verdicts, and how do candidate-set size, token usage, and cost correlate with success? This RQ is inspired by recent agentic SZZ analyses that report tool calls, tokens, cost, and behavior distributions [4], [6]. Result table: [*NEEDS EXPERIMENT*].

E. Pilot Status

The local SystemDesign directory contains current pilot artifacts over a 32-case subset. These artifacts are useful for debugging the paper pipeline and metric implementation, but they are not the final ICSE results and should not be compared directly against full 1,128-CVE paper baselines. Any pilot table copied into the paper must be labeled as a pilot and kept separate from the main evaluation [*NEEDS EXPERIMENT*].

VII. Threats to Validity

Construct validity. The affected-version task has both exact-set and partial-version aspects. We therefore use CVE-level Accuracy, CVE-level No-Miss Ratio, and version-level precision/recall/F1 rather than relying on a single metric. Resource metrics such as tokens and cost must be reported separately because an agent-based method can trade accuracy for runtime or API cost.

Internal validity. VulnVersiondepends on correct patch-family filtering, root-cause evidence extraction, release-line modeling, and tag verdicts. Errors in any stage can propagate to the final affected-version set. The planned ablations in RQ3 are necessary before attributing improvements to a specific component. Current method details and results remain [*NEEDS EXPERIMENT*].

External validity. The primary dataset is C/C++ focused and covers nine open-source projects. Results may not transfer directly to ecosystems with different release practices, package managers, or vulnerability disclosure conventions. We will avoid claims about other languages until additional datasets are evaluated.

Baseline validity. The baseline study provides the most relevant comparison point, but local reproduction depends on artifact availability, environment compatibility, and exact data splits. If we use paper-reported baseline values, we must clearly label them as paper-reported. If we reproduce them, we must release scripts and environment details [*NEEDS ARTIFACT*].

Citation and table validity. Several local reference analyses mark table values, figure crops, and citation metadata as requiring repair or verification. Therefore, all exact numeric claims taken from extracted tables must be checked against the original PDFs before submission [*NEEDS TABLE REPAIR*] [*NEEDS CITATION VERIFICATION*].

Agent validity. If VulnVersionuses an LLM or agent backend, verdicts may depend on prompt design, context compression, model version, randomness, and tool-output formatting. We will log model/backend information, tokens, cost, latency, failure rate, and replay artifacts where possible. Any claim about agent memory or self-evolution remains out of scope until supported by ablations [*NEEDS EXPERIMENT*].

Ethics and misuse. Affected-version identification can help defenders prioritize patching, but it can also reveal which deployed versions are likely vulnerable. Prior patch-search work explicitly surfaces this dual-use risk [9]. The paper should emphasize defensive use, benchmark-level evaluation, and responsible disclosure if new vulnerable versions are discovered [*NEEDS EVIDENCE*].

VIII. Related Work

A. Direct Affected-Version Identification

The closest work is the large-scale benchmark study of vulnerability-affected versions [1]. It defines the task, constructs the main dataset used in this paper, and evaluates direct tools under both CVE-level and version-level metrics. Earlier work such as V-SZZ [13] explicitly connects SZZ-style vulnerability-inducing commit localization to affected-version range identification. TDSC's patch-and-developer-log based approach [14] further highlights the importance of branch-aware reasoning and lower-bound recall when version metadata is incomplete.

Recent LLM-assisted vulnerable-version methods, including Vercation [15], move closer to semantic patch understanding and context-aware version judgment. VulnVersionshares their motivation but emphasizes a stricter separation between deterministic repository planning and agent-assisted tag verdicts.

B. Tracing-Based Commit Localization

SZZ-style methods identify bug- or vulnerability-inducing commits and are often used as upstream components for affected-version inference. VCCFinder [16], Lifetime [17], SEM-SZZ [3], TC-SZZ [18], and LLM4SZZ [2] represent increasingly refined tracing pipelines. AgentSZZ [4], How and Why Agents Can Identify Bug-Introducing Commits [6], Beyond Blame [5], and patch-pattern differential analysis [19] show that repository exploration, pattern reasoning, and knowledge-graph or agentic inference can improve commit-level localization. However, their primary output remains a commit or commit set, whereas VulnVersiontargets released-version state verification.

C. Recurring Vulnerability and Clone-Based Detection

Matching-based and recurring vulnerability detection systems provide useful per-version checking components. ReDeBug [9], VUDDY [12], MOVERY [8], V1SCAN [10], FIRE [7], and VULTURE [11] detect vulnerable code clones, patch-related signatures, or reused vulnerable components. These systems demonstrate that patch-derived code evidence can be searched across versions, but they also expose the brittleness of exact or near-exact code resemblance when patches are add-only, multi-file, or semantically transformed.

VulnVersiontreats these systems as evidence providers and baseline families rather than complete solutions to the affected-version problem. This distinction is important: a clone, hash, signature, or taint path can support a tag verdict, but the final

claim still asks whether the vulnerable condition holds in a released repository state.

D. Exploitability, PoC Migration, and Cross-Version Validation

The local replication corpus also contains work on cross-version exploitability and PoC migration, including AEM [20], ARVO [21], Diffploit [22], PoC migration [23], library vulnerability migration [24], PoCGen [25], PwnGPT [26], SemFuzz [27], SyzBridge [28], real-world PoC usability [29], and API-equivalence vulnerability detection [30]. These papers are not all yet analyzed in the local `Paper/reference` corpus, so this paragraph should remain citation-placeholding until their summaries are verified [*NEEDS CITATION VERIFICATION*].

E. Agent Memory and Skill Evolution

VulnVersion's long-term design includes typed memory, verifier-gated self-evolution, and reusable skills. This direction is related to agent memory surveys and systems [31]–[34], as well as skill evolution and benchmarking work [35]–[39]. In the current paper, these references motivate future extensions rather than completed evaluation claims.

IX. Conclusion

This draft frames VulnVersionas a repository-grounded approach to identifying CVE-affected released versions from patch evidence. The central position is that affected-version identification should be treated as per-tag vulnerability state verification, not only as vulnerability-inducing commit localization or patch-signature matching. The current paper skeleton establishes the task definition, motivation, related-work structure, method placeholder, and evaluation plan around the "How Far Are We?" benchmark. Final method details, full experimental results, ablations, resource measurements, and verified citation metadata remain required before this can become an ICSE submission [*NEEDS EXPERIMENT*].

References

[1] X. Chen, C. Liu, J. Cao, Y. Xiao, X. Cai, Y. Li, J. Shi, T. Sun, H. Chen, and W. Huo, "Vulnerability-Affected Versions Identification: How Far Are We?" 2025. [Online]. Available: https://arxiv.org/abs/2509.03876

[2] L. Tang, J. Liu, Z. Liu, X. Yang, and L. Bao, "LLM4SZZ: Enhancing SZZ algorithm with context-enhanced assessment on large language models," 2025. [Online]. Available: https://arxiv.org/abs/2504.01404

[3] L. Tang, C. Ni, Q. Huang, and L. Bao, "Enhancing bug-inducing commit identification: A fine-grained semantic analysis approach," *IEEE Transactions on Software Engineering*, vol. 50, no. 11, pp. 3037–3052, 2024. [Online]. Available: https://doi.org/10.1109/TSE.2024.3468296

[4] Y. Lyu, J. Shi, H. J. Kang, R. Widyasari, J. He, Y. Niu, C. Yang, J. Chen, Z. Yang, J. Lawall, and D. Lo, "AgentSZZ: Teaching the LLM agent to play detective with bug-inducing commits," 2026. [Online]. Available: https://arxiv.org/abs/2604.02665

[5] Y. Shi, H. Li, B. Adams, and A. E. Hassan, "Beyond Blame: Rethinking SZZ with knowledge graph search," 2026. [Online]. Available: https://arxiv.org/abs/2602.02934

[6] N. Risse and M. Böhme, "How and why agents can identify bug-introducing commits," 2026. [Online]. Available: https://arxiv.org/abs/2603.29378

[7] S. Feng, Y. Wu, W. Xue, S. Pan, D. Zou, Y. Liu, and H. Jin, "FIRE: Combining Multi-Stage filtering with taint analysis for scalable recurring vulnerability detection," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 1867–1884. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/feng-siyue

[8] H. Han, S. Kim, H. Kwon, H. Jin, H. Lee, and H. Lee, "Movery: A precise approach for modified vulnerable code clone discovery from modified open-source software components," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3037–3053. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/kang

[9] J. Jang, A. Agrawal, and D. Brumley, "ReDeBug: Finding unpatched code clones in entire OS distributions," in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 48–62. [Online]. Available: https://doi.org/10.1109/SP.2012.13

[10] Y. Dong, W. Guo, Y. Chen, X. Xing, Y. Zhang, and G. Wang, "V1SCAN: Discovering 1-day vulnerabilities in reused C/C++ open-source software components using code classification techniques," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, 2023, pp. 6541–6558. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/dong

[11] S. Xu, J. Dong, W. Cai, J. Li, A. Shaghaghi, N. Sun, and S. Ma, "Enhancing security in third-party library reuse: Comprehensive detection of 1-day vulnerability through code patch analysis," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '25. Internet Society, 2025. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/enhancing-security-in-third-party-library-reuse-comprehensive-detection-of-1-day-vul

[12] S. Kim, S. Woo, H. Lee, and H. Oh, "VUDDY: A scalable approach for vulnerable code clone discovery," in *2017 IEEE Symposium on Security and Privacy*. IEEE, 2017, pp. 595–614. [Online]. Available: https://doi.org/10.1109/SP.2017.62

[13] Y. He, Y. Wang, S. Zhu, W. Wang, Y. Zhang, A. Yu, and Q. Li, "V-SZZ: Automatic identification of version ranges affected by CVE vulnerabilities," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: ACM, 2022, pp. 2352–2364. [Online]. Available: https://doi.org/10.1145/3510003.3510163

[14] Y. He, Y. Wang, S. Zhu, W. Wang, Y. Zhang, Q. Li, and A. Yu, "Automatically identifying CVE affected versions with patches and developer logs," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 905–919, 2024. [Online]. Available: https://doi.org/10.1109/TDSC.2023.3264567

[15] Y. Cheng, T. Zhang, L. K. Shar, S. Yang, C. Dong, D. Lo, S. Lv, Z. Shi, and L. Sun, "VERCATION: Precise vulnerable open-source software version identification based on static analysis and LLM," *IEEE Transactions on Software Engineering*, vol. 52, no. 2, pp. 376–394, 2026. [Online]. Available: https://doi.org/10.1109/TSE.2025.3599581

[16] F. Yamaguchi, F. Lindner, and K. Rieck, "VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '15. New York, NY, USA: ACM, 2015, pp. 426–437. [Online]. Available: https://doi.org/10.1145/2810103.2813604

[17] Y. Zhang, E. Alégroth, R. Krishna, H. C. Gall, and C. Tantithamthavorn, "How long do vulnerabilities live in the code? A large-scale empirical measurement study on FOSS vulnerability lifetimes," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 359–376. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/zhang-yuxing

[18] Y. Lyu, H. J. Kang, R. Widyasari, J. Lawall, and D. Lo, "Evaluating SZZ implementations: An empirical study on the Linux kernel," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2219–2239, 2024. [Online]. Available: https://doi.org/10.1109/TSE.2024.3406718

[19] Q. Guo and Y. He, "Accurate identification of the vulnerability-introducing commit based on differential analysis of patching patterns," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '26. Internet Society, 2026. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/auto-draft-516/

[20] Z. Jiang, Y. Zhang, J. Xu, X. Sun, Z. Liu, and M. Yang, "AEM: Facilitating cross-version exploitability assessment of Linux kernel vulnerabilities," in *2023 IEEE Symposium on Security*

and Privacy. IEEE, 2023, pp. 2122–2137. [Online]. Available: https://doi.org/10.1109/SP46215.2023.10179286

[21] X. Mei, P. S. Singaria, J. D. Castillo, H. Xi, A. Benchikh, T. Bao, R. Wang, Y. Shoshitaishvili, A. Doupé, H. Pearce, and B. Dolan-Gavitt, "ARVO: Atlas of reproducible vulnerabilities for open source software," 2024. [Online]. Available: https://arxiv.org/abs/2408.02153

[22] Z. Chen, Z. Xue, J. Zhou, X. Hu, X. Xia, and X. Yang, "Diffploit: Facilitating cross-version exploit migration for open source library vulnerabilities," 2025. [Online]. Available: https://arxiv.org/abs/2511.12950

[23] J. Dai, Y. Zhang, H. Xu, H. Lyu, Z. Wu, X. Xing, and M. Yang, "Facilitating vulnerability assessment through PoC migration," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. New York, NY, USA: ACM, 2021, pp. 3300–3317. [Online]. Available: https://doi.org/10.1145/3460120.3484594

[24] T. Zhang, Z. Zhang, D. Zhang, X. Luo, J. Ming, and M. Yang, "Exploiting library vulnerability via migration based automating test generation," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, ser. ICSE '24. ACM, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639583

[25] D. Simsek, A. Eghbali, and M. Pradel, "PoCGen: Generating proof-of-concept exploits for vulnerabilities in npm packages," 2025. [Online]. Available: https://arxiv.org/abs/2506.04962

[26] T. Zhang, T. Xia, C. Li, C. Wei, Y. Cai, L. Zhang, and X. Luo, "PwnGPT: Automatic exploit generation based on large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 11 455–11 479. [Online]. Available: https://aclanthology.org/2025.acl-long.562/

[27] W. You, P. Zong, K. Chen, X. Wang, X. Liao, P. Bian, and B. Liang, "SemFuzz: Semantics-based automatic generation of proof-of-concept exploits," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: ACM, 2017, pp. 2139–2154. [Online]. Available: https://doi.org/10.1145/3133956.3134085

[28] Z. Lin, Y. Ma, Z. Lin, X. Luo, H. Chen, and D. Mu, "SyzBridge: Bridging the gap in exploitability assessment of Linux kernel bugs in the Linux ecosystem," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '24. Internet Society, 2024. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/syzbridge-bridging-the-gap-in-exploitability-assessment-of-linux-kernel-bugs-in-the-linux-ecosystem/

[29] W. Dang, K. Li, S. Chen, Z. Zhuo, L. Zhang, and Z. Liu, "Real-world usability of vulnerability proof-of-concepts: A comprehensive study," 2025. [Online]. Available: https://arxiv.org/abs/2510.18448

[30] T. Chen, Z. Wang, L. Li, D. Li, Z. Li, X. Chang, P. Bian, G. Liang, Q. Wang, and T. Xie, "Detecting functionality-specific vulnerabilities via retrieving individual functionality-equivalent APIs in open-source repositories," in *39th European Conference on Object-Oriented Programming*, ser. LIPIcs, vol. 333. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 6:1–6:27. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2025.6

[31] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan, "Memory in the age of AI agents," 2025. [Online]. Available: https://arxiv.org/abs/2512.13564

[32] R. Fang, Y. Liang, X. Wang, J. Wu, S. Qiao, P. Xie, F. Huang, H. Chen, and N. Zhang, "Memp: Exploring agent procedural memory," 2025. [Online]. Available: https://arxiv.org/abs/2508.06433

[33] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang, "MemSkill: Learning and evolving memory skills for self-evolving agents," 2026. [Online]. Available: https://arxiv.org/abs/2602.02474

[34] N. Belle, D. Barnes, A. Amayuelas, I. Bercovich, X. E. Wang, and W. Wang, "Agents of change: Self-evolving LLM agents for strategic planning," 2025. [Online]. Available: https://arxiv.org/abs/2506.04651

[35] Y. Yang, J. Li, Q. Pan, B. Zhan, Y. Cai, L. Du, J. Zhou, K. Chen, Q. Chen, X. Li, B. Zhang, and L. He, "AutoSkill: Experience-driven lifelong learning via skill self-evolution," 2026. [Online]. Available: https://arxiv.org/abs/2603.01145

[36] H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, X. Liu, X. Li, and P. S. Yu, "CoEvoSkills: Self-evolving agent skills via co-evolutionary verification," 2026. [Online]. Available: https://arxiv.org/abs/2604.01687

[37] P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, Z. Zheng, C. Xie, and H. Yao, "SkillRL: Evolving agents via recursive skill-augmented reinforcement learning," 2026. [Online]. Available: https://arxiv.org/abs/2602.08234

[38] X. Li, W. Chen, Y. Liu, S. Zheng, X. Chen, Y. He, Y. Li, B. You, H. Shen, J. Sun, S. Wang, Q. Zeng, D. Wang, X. Zhao, Y. Wang, R. B. Chaim, Z. Di, Y. Gao, J. He, Y. He, L. Jing, L. Kong, X. Lan, J. Li, S. Li, Y. Li, Y. Lin, X. Liu, X. Liu, H. Lyu, Z. Ma, B. Wang, R. Wang, T. Wang, W. Ye, Y. Zhang, H. Xing, Y. Xue, S. Dillmann, and H. chung Lee, "SkillsBench: Benchmarking how well agent skills work across diverse tasks," 2026. [Online]. Available: https://arxiv.org/abs/2602.12670

[39] J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang, "Trace2Skill: Distill trajectory-local lessons into transferable agent skills," 2026. [Online]. Available: https://arxiv.org/abs/2603.25158